Geospatial Web Lab: Project Summary

Portfolio preparation edition | 29 September 2026

Purpose and scope

Geospatial Web Lab is an AI-assisted learning project containing five working full-stack applications. It connects Ruby on Rails, JavaScript, React, PostgreSQL/PostGIS, background jobs, and interactive mapping through complete user workflows.

The project uses synthetic business records and geometry. Evidence covers implementation and local verification, with no claims of adoption, production traffic, business savings, or operational deployment.

Five applications

1. **Civic Works service-request manager.** Reporters create mapped requests; staff assign and advance their status. Linked map and register filters support review. GeoJSON imports handle row errors and duplicate submissions; background CSV exports produce downloadable reports.
2. **Geospatial data-quality portal.** Users upload GeoJSON, inspect structural and PostGIS topology findings, and review accepted features on a map. Staff approve publication snapshots, explicitly acknowledging excluded records. Approved exports preserve their content, reviewer, feature count, and SHA-256 digest.
3. **Fleet and geofence monitor.** Ten synthetic vehicles follow replayable routes. Users inspect positions, trails, speeds, and geofence transitions. Staff control replay and zones. Sequence checks reject older telemetry; spatial rules count geofence boundaries as inside.
4. **Parcel scenario explorer.** Users select mapped parcels and compare asynchronous calculations of floor area, unit estimates, and open space. Exports retain assumptions and parcel geometry. Simplified planning formulas include synthetic height-limit warnings.
5. **3D infrastructure inspections.** A CesiumJS corridor displays eight synthetic assets with separate base elevations and structure heights. Users report observations; staff resolve or reopen them with an audit history. Background preprocessing produces a chart, sample table, and downloadable geodesic profile.

Architecture and skills demonstrated

Each application has a separate Rails API, database, Redis queue configuration, and worker configuration. The npm workspace shares React components, authentication helpers, and ArcGIS maps. Frontend proxies connect browsers to Rails; Action Cable carries authenticated change notifications.

Rails applies permissions and validation before database writes. PostGIS supplies spatial filtering, geography-based distance and area, geometry validity checks, and geodesic interpolation. Sidekiq processes imports, exports, validation, replay, and calculations. React refreshes authoritative API state after live notifications, with polling and reconnection recovery.

Learning area	Implemented evidence
Rails, JavaScript, and React	Authenticated CRUD, server validation, record ownership, and shared interface components
Interactive interfaces	Linked maps and tables, charts, job progress, live status changes, and responsive layouts
Geospatial integration	ArcGIS 2D views, Cesium 3D assets, and indexed PostGIS operations
Backend processing	Transactions, row/advisory locks, bounded datasets, retry handling, and background workers
Integration and deployment	API/proxy contracts, runtime scripts, container definitions, migrations, and environment configuration
Automated reliability checks	Rails tests, React component tests, real browser workflows, and runtime regressions

Engineering decisions and audit corrections

Keep committed results independent of notifications. Notification outages previously could make successful database operations appear to fail. Notifications now fail separately from durable writes; clients recover current state. Queue rejection and late acknowledgement paths preserve work already started or completed.

Protect concurrent work and account boundaries. Optimistic locking blocks stale request and inspection edits. Job generations and revisions prevent obsolete work from replacing newer results. Frontend sequencing guards against delayed reads, uploads, and saves. Revocable login tokens invalidate copied cookies after logout, while cross-tab changes remove the previous account's workspace and draft. Repeated failed sign-ins are throttled. Automatic retries for imports, exports, and validation are bounded: imports and exports make up to three Active Job attempts, while validation retries only connection-establishment failures. Sidekiq's additional retries are disabled for those jobs and scenario calculations, so runs that remain failed use recovery actions in the app.

Make spatial assumptions explicit. Polygon winding is normalized for map rendering without changing source geometry. Profile samples follow geodesics across the antimeridian. Parcel areas use PostGIS geography. Approved dataset versions have database-level immutability; completed calculations and profiles preserve source snapshots for reproducible exports.

Recorded verification

The verification record is dated **2026-09-29**. These figures describe earlier checks, not a new test run for this document.

Check	Recorded result
Backend	109 tests, 690 assertions; no failures, errors, or skips
Frontend	44 tests passed
Browser workflows	13 scenarios passed against real local services
Production frontend builds	All five passed

A local run on **2026-10-17**, after the October review and issue fixes, recorded 159 backend tests (1126 assertions), 58 root frontend tests and 81 standalone tests passing, with ESLint and Brakeman in CI; browser workflows and container builds were left to CI.

Additional checks cover production eager loading, restricted-role migrations, Compose configuration, process handling, and backup restore. VERIFICATION.md records evidence boundaries; AUDIT.md and AUDIT_SECOND_PASS.md describe corrected defects.

Current delivery boundaries

Applications ran directly in WSL. Docker builds and execution were not verified locally because the engine was unavailable; all five stacks later built and passed a container smoke test on GitHub Actions. Remote CI status is available in GitHub Actions, separately from the recorded local checks. The full-stack applications have no public deployment; the standalone browser editions are published at https://efkopru.github.io/geospatial-web-lab/. Frontend builds establish compilation, not deployment.

ArcGIS basemaps and SDK assets require internet access. Cesium uses synthetic elevations and local assets without a terrain-service token. The 3D view requires WebGL. Imports and histories are bounded. Recorded baselines cover the standalone editions and the full-stack APIs and workers. Both are single-machine measurements on synthetic data, not capacity guarantees.

Copy-ready portfolio text

> Geospatial Web Lab is an AI-assisted learning project comprising five Rails and React applications with PostgreSQL/PostGIS, ArcGIS, and CesiumJS. It explores mapped service requests, GeoJSON quality review, simulated fleet monitoring, parcel calculations, and 3D inspections. The implementation includes authenticated APIs, background jobs, live updates, reproducible exports, and regression tests. Recorded local verification covers 109 backend tests, 44 frontend tests, 13 browser scenarios, and five frontend builds. Standalone browser editions are published as a static demo; the full-stack applications are not publicly hosted.

- Implemented workflows connect spatial databases, background processing, and linked map interfaces.
- Two audit passes addressed session revocation, asynchronous failure handling, stale edits, and spatial edge cases.
- Synthetic datasets and documented verification distinguish demonstrated behavior from deployment and scale claims.

Portfolio release artifacts

- Capture a short workflow video and current screenshots for each application.

- Select a concise code walkthrough explaining one spatial operation and one tested failure case.
- Record the repository revision and successful remote CI run before presenting CI status as evidence.
- Verify container execution and hosting before adding a live-demo link.

Use README.md for setup and architecture, and HOW_TO_USE.md for operating the applications.